

操作系统原理

第三章：操作系统结构

洪明坚

重庆大学软件学院

February 19, 2019

1 Operating system structures

- Simple structure
- Layered structure
- Micro-kernel
- Virtual machine

2 Operating system design

- Policy and mechanism
- Implementation

Outline

1 Operating system structures

- Simple structure
- Layered structure
- Micro-kernel
- Virtual machine

2 Operating system design

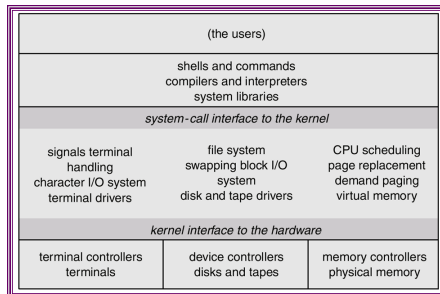
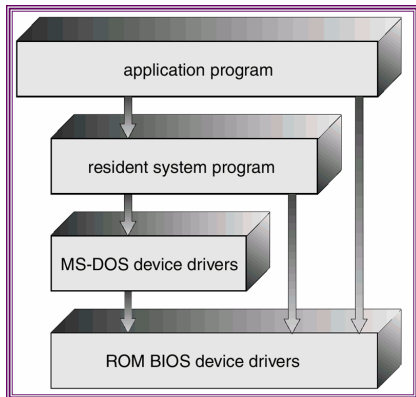
- Policy and mechanism
- Implementation

System structure

- A system as large and complex as a modern operating system must be engineered carefully if it's to function properly and be modified easily.
- A common approach is to partition the task into small components, rather than have one monolithic system.
 - Each of these components should be a well-defined portion of the system, with carefully defined inputs, outputs and function.
- How does the system designer organize these components?
 - Simple structure (or no structure)
 - Layered structure
 - Micro-kernel
 - Virtual machine

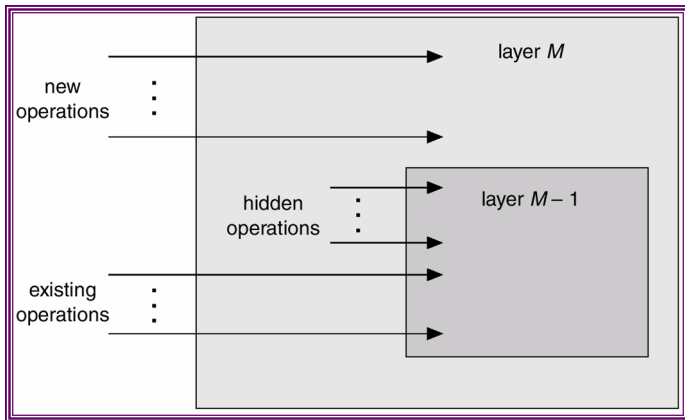
Simple structure

- Many systems do not have a well-defined structure.
 - They started as small, simple and limited system, and then evolved into a complex one.
- Examples: MS-DOS and Unix



Layered structure (1/4)

- The operating system is broken up into a number of layers, each built on top of lower layers.



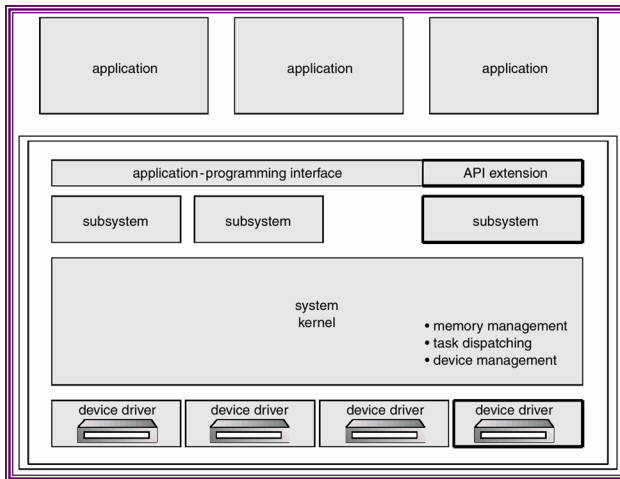
Layered structure (2/4)

- Example 1:
 - The **THE** operating system by Dijkstra.

Layer	Function
5	The operator
4	User programs
3	Input/output management
2	Operator-process communication
1	Memory and drum management
0	Processor allocation and multiprogramming

Layered structure (3/4)

- Example 2:
 - The IBM OS/2 operating system

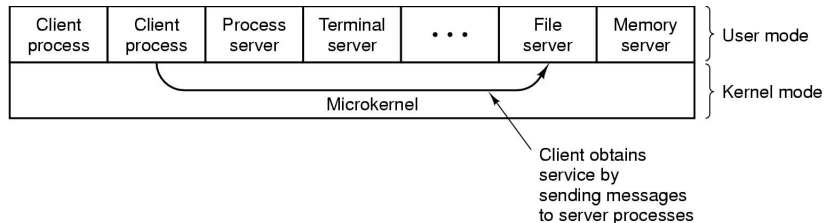


Layered structure (4/4)

- The major difficulty with the layered structure involves
 - ① careful definition of the layers and
 - ② less efficient.

Micro-kernel (1/2)

- As the Unix operating system expanded, the kernel became large and difficult to manage.
 - The **micro-kernel** approach structures the operating system by removing all non-essential components from the kernel and implementing them as system- and user-level programs.
- Which components should remain in the micro-kernel?
 - CPU management
 - Memory management
 - Communication facility



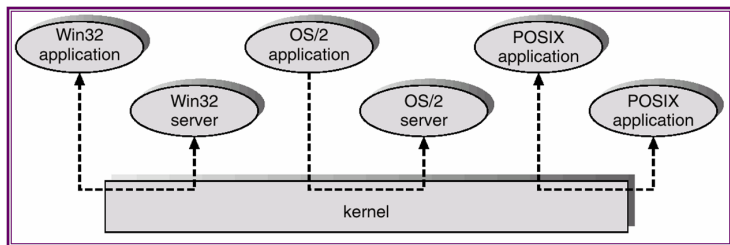
Micro-kernel (2/2)

- Example 1

- The open source **Mach** from Carnegie Mellon University.
 - Used as the kernel of the Apple macOS/iOS/tvOS/watchOS.

- Example 2

- Microsoft Windows NT/XP



Questions

- Any questions?

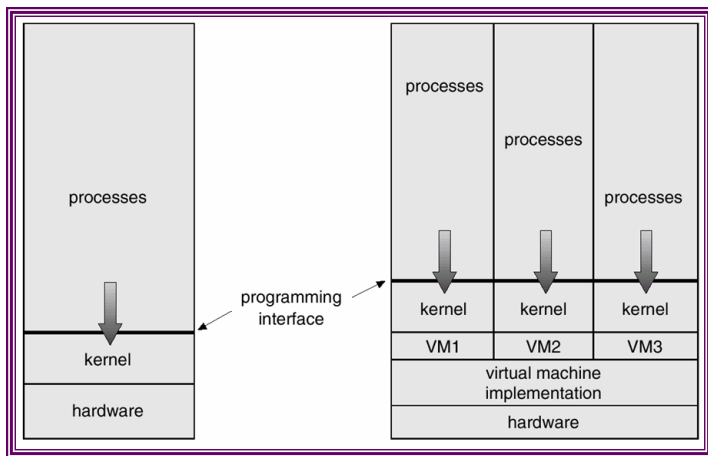


Virtual machine (1/5)

- If we take a step further from micro-kernel,
 - The low-level real hardware is “cloned” into several identical **virtual machines**.
 - That is, virtual machine provides an interface identical to the underlying bare hardware.
- Then, the operating system functionality is built on top of the virtual machines.

Virtual machine (2/5)

- Virtual machine structure



- Usually, virtual machines can be classified into two kinds:
 - **Hardware virtual machines**
 - VM/370 from IBM
 - **Xen** from Cambridge University
 - **KVM** (Kernel-based Virtual Machine) for Linux
 - **VMware**, **VirtualBox**, **QEMU**, **Bochs**, etc
 - **Application virtual machines** - a piece of computer software that isolates the application being used by the user from the computer.
 - Example: Sun microsystem's Java Virtual Machine (JVM).

Virtual machine (4/5)

- Hardware virtual machines can be implemented in three ways:
 - **Emulation** - the virtual machine simulates the complete hardware, allowing an unmodified operating system for a completely different CPU to run.
 - Examples: VMware, VirtualBox, QEMU, Bochs, etc.
 - **Paravirtualization** - the virtual machine does not simulate hardware but instead offers a special API that requires operating system modifications to run.
 - Examples: **Xen**
 - **Native (Full) virtualization** - the virtual machine only partially simulates enough hardware to allow an unmodified operating system to run in isolation, but the **guest** operating system must be designed for the same type of CPU
 - Examples: KVM and VM/370

- Advantage and disadvantage of virtual machine
 - The virtual-machine concept provides complete protection of system resources since each virtual machine is isolated from all other virtual machines.
 - This isolation, however, permits no direct sharing of resources
 - A virtual-machine system is a perfect vehicle for operating-systems research and development.
 - The virtual machine concept is difficult to implement due to the effort required to provide an **exact** duplicate to the underlying machine.

Questions

- Any questions?



Outline

1 Operating system structures

- Simple structure
- Layered structure
- Micro-kernel
- Virtual machine

2 Operating system design

- Policy and mechanism
- Implementation

- Design goals

User goals operating system should be convenient to use, easy to learn, reliable, safe, and fast.

System goals operating system should be easy to design, implement, and maintain, as well as flexible, reliable, error-free, and efficient.

- **Policies - what to do**

- For example, users should not be able to read other users' files.

- **Mechanisms - how to do**

- For example, file permissions are checked on *open* system call.

- The **separation** of policy from mechanism is a very important principle

- It allows maximum flexibility if policy decisions are to be changed later.

The operating system implementation

- Traditionally written in assembly language, operating systems can now be written in higher-level languages.
- Code written in a high-level language:
 - can be written faster.
 - is more compact.
 - is easier to understand and debug.
 - is easier to port to some other hardware.

Questions

- Any questions?

